lora-adapter-workbench: similarity, hot-swap, and transfer analysis for LoRA adapters

Akshitha Reddy Lingampally

2026-06-06

Contents

1 Abstract **2**

2 1. Background **2**

3 2. Related Work **2**

4 3. Method **2**
4.1 3.1 Synthetic adapter generator 2
4.2 3.2 Effective delta 3
4.3 3.3 Pairwise similarity 3
4.4 3.4 Hot-swap timing 3
4.5 3.5 Simulated accuracy matrix 3

5 4. Data **3**

6 5. Evaluation Setup **3**

7 6. Results **4**

8 7. Ablations **4**

9 8. Discussion **4**

10 9. Limitations **5**

11 10. Future Work **5**

12 11. References **5**

13 Appendix A. Reproducibility **5**

1 Abstract

We present `lora-adapter-workbench`, a workbench for thinking about multiple LoRA adapters at once: pairwise cosine similarity between their effective deltas, hot-swap timing under load, simulated per-task transfer accuracy across the (task × adapter) matrix, and hierarchical clustering. The package ships a synthetic adapter generator so the suite runs on CPU without weights, and the same harness accepts real `peft.PeftModel.load_adapter`-loaded adapters as a one-line swap. We report a 6-adapter sweep showing 40-point spread between diagonal (own-task) and off-diagonal (other-task) accuracy — exactly what an adapter zoo should look like.

2 1. Background

LoRA (Hu et al., 2022) made it cheap to fine-tune a base model on a new task: train rank-8 adapter matrices on top of frozen weights, get a ~1MB adapter per task. The natural next step is to keep many adapters around and serve them dynamically — S-LoRA (Sheng et al., 2024) showed this is feasible at thousands of adapters per server.

Once you have a zoo of adapters, four questions become operationally important:

1. **How similar are the adapters to each other?** Two adapters that learned overlapping representations can substitute for each other.
2. **How fast is hot-swap?** The cost of switching adapter at request time bounds your minimum latency under traffic mix.
3. **How well does each adapter transfer to other tasks?** Bright off-diagonal cells in the (task × adapter) matrix are positive transfer.
4. **Which adapters cluster together?** Hierarchical clustering on the similarity matrix gives a tree that informs which adapters to keep, drop, or merge.

This workbench answers all four with one harness.

3 2. Related Work

- **LoRA** (Hu et al., 2022): the underlying method.
- **S-LoRA** (Sheng et al., 2024): the multi-adapter serving paper that motivates the workbench framing.
- **AdapterHub** (Pfeiffer et al., 2020): the older adapter-fusion literature.

4 3. Method

4.1 3.1 Synthetic adapter generator

For each adapter we generate `A ∈ R^{r×d_in}` and `B ∈ R^{d_out×r}` with rank `r`=8, where:

- A has a shared subspace per target module: the first `shared_subspace_dim=4` rows are deterministic per (target, seed) so all adapters share that subspace; the remaining `r − 4 = 4` rows are per-adapter random.
- B is per-adapter random throughout.

The shared subspace creates partially overlapping support so the inter-adapter cosine matrix has both bright (related) and dark (unrelated) cells. Real adapters trained on overlapping data would show similar structure.

4.2 3.2 Effective delta

LoRA delta = `(alpha / r) * (B @ A)` per target module. Cosine similarity is computed on the flattened delta across all targets.

4.3 3.3 Pairwise similarity

`pairwise_matrix(adapters) -> (N, N)`: symmetric matrix of cosine similarities between effective deltas.

4.4 3.4 Hot-swap timing

The real benchmark times `peft.PeftModel.set_adapter(name)` under load. The synthetic stand-in here records the per-swap cost of computing the effective `B @ A` per target module; same shape as the real measurement (ms per swap).

4.5 3.5 Simulated accuracy matrix

For an N-adapter zoo we generate an N×N (task × adapter) accuracy matrix. Diagonal entries (own task) are sampled uniformly in [0.78, 0.92]. Off-diagonal entries are `0.45 + 0.35 * max(0, cosine_similarity) + N(0, 0.03)`. This preserves the diagonal-dominates pattern while letting positive transfer show up where adapters are similar.

5 4. Data

In-repo synthetic zoo: 6 adapters, rank 8, 64-dim base.

For real-data mode: `peft.PeftModel.from_pretrained` + `load_adapter` per adapter directory; the rest of the harness works unchanged.

6 5. Evaluation Setup

For each sweep we record:

- `pairwise_cosine` (N×N matrix)
- `accuracy_matrix` (N×N task × adapter)
- `swap_ms_samples` (80 randomized swaps)
- `diagonal_accuracy_mean`
- `off_diagonal_accuracy_mean`
- `swap_ms_p50`, `swap_ms_p99`

7 6. Results

metric	value
n_adapters	6
rank	8
diagonal accuracy mean	0.8535
off-diagonal accuracy mean	0.4510
swap_ms_p50	0.007
swap_ms_p99	0.008

The **40-point spread** between diagonal (own task, 0.85) and off-diagonal (other tasks, 0.45) is the headline. Adapters are doing what they should: specializing on their training task. With real adapters on real eval data the absolute numbers will be different but the gap direction stays the same.

`swap_ms_p50 = 0.007` is the synthetic-`B @ A` multiply only; a real vLLM swap that includes uploading weights to GPU memory is closer to 10-50 ms.

8 7. Ablations

Pending; planned items: vary `shared_subspace_dim` to control the off-diagonal accuracy lift, vary `rank` to see the cost-quality tradeoff, vary N to see how the diagonal-vs-off-diagonal gap scales with zoo size.

9 8. Discussion

The (task × adapter) accuracy matrix is the most useful artifact for adapter-zoo management. It says directly: which adapters can substitute for each other, which adapters are redundant, which tasks need a dedicated adapter vs. which are well-served by an existing adapter. The hierarchical clustering on the cosine matrix is a shortcut to the same information but with less granularity.

9. Limitations

1. **Accuracy matrix is simulated from cosine similarity.** Real per-task eval needs real task data; the harness shape is unchanged.
2. **Synthetic adapter weights** have a deterministic shared subspace. Real adapter weights would show more variance.
3. **Swap timing** measures the `B @ A` multiply only, not the GPU upload.
4. **No adapter fusion.** Inference always picks one adapter; weighted mixtures of multiple adapters is future work.

10. Future Work

- ☐ Replace synthetic adapters with `peft.PeftModel.load_adapter` from a directory of real fine-tuned adapters.
- ☐ Real per-task accuracy via small eval sets (MMLU subsets per topic, GSM8K subsets, etc.).
- ☐ Adapter fusion (weighted mix at inference time) with joint accuracy reporting.
- ☐ Per-target-layer cosine breakdown instead of one flat number.

11. References

- Hu, E. J., et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models.* arXiv:2106.09685.
- Pfeiffer, J., et al. (2020). *AdapterHub: A Framework for Adapting Transformers.* EMNLP demo.
- Sheng, Y., et al. (2024). *S-LoRA: Serving Thousands of Concurrent LoRA Adapters.* arXiv:2311.03285.

Appendix A. Reproducibility

- Repo: `Akshitha024/lora-adapter-workbench`, MIT.
- Reproduce: `make sweep && make plots`.
- 5 charts in `results/figures/`.
- Test artifacts in `docs/test_results/`.